



University
of Exeter

COMM113

Deep Learning – Week 4

Regularisations II

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Review

Deep Learning

- [Table of Contents](#)
- [Acknowledgements](#)
- [Notation](#)
- [1 Introduction](#)
- [Part I: Applied Math and Machine Learning Basics](#)
 - [2 Linear Algebra](#)
 - [3 Probability and Information Theory](#)
 - [4 Numerical Computation](#)
 - [5 Machine Learning Basics](#)
- [Part II: Modern Practical Deep Networks](#)
 - [6 Deep Feedforward Networks](#)
 - [7 Regularization for Deep Learning](#)
 - [8 Optimization for Training Deep Models](#)
 - [9 Convolutional Networks](#)
 - [10 Sequence Modeling: Recurrent and Recursive Nets](#)
 - [11 Practical Methodology](#)
 - [12 Applications](#)
- [Part III: Deep Learning Research](#)
 - [13 Linear Factor Models](#)
 - [14 Autoencoders](#)
 - [15 Representation Learning](#)
 - [16 Structured Probabilistic Models for Deep Learning](#)
 - [17 Monte Carlo Methods](#)
 - [18 Confronting the Partition Function](#)
 - [19 Approximate Inference](#)
 - [20 Deep Generative Models](#)

Please read Part 1 of the Textbook "Deep Learning"

[Deep Learning](https://www.deeplearningbook.org/)

<https://www.deeplearningbook.org/>

Batch Normalisation

```
# Download and load training data  
# root: where to save the data  
# train: whether to use training split  
# download: download if not already present  
train_dataset = torchvision.datasets.FashionMNIST(  
    root='./data',  
    train=True,  
    download=True,  
    transform=transform  
)
```

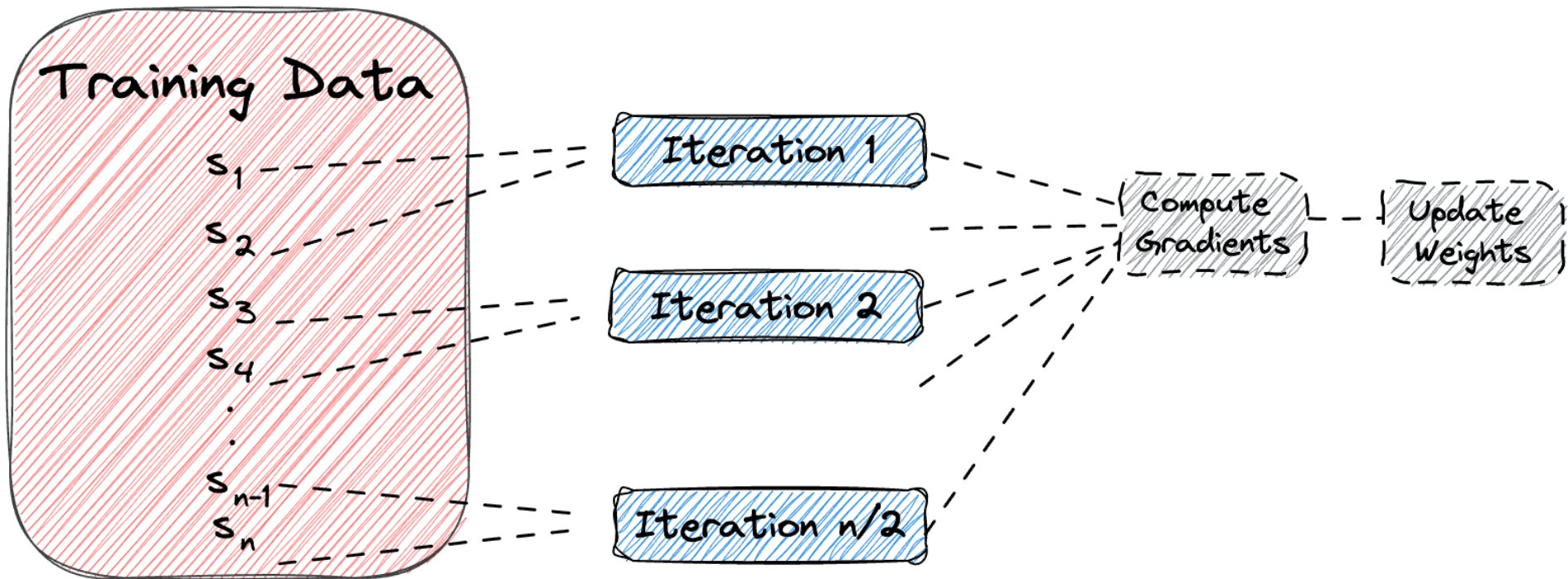
Load training dataset

```
# Create data loaders for batching  
# batch_size: number of samples per batch  
# shuffle: randomize order for training  
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)  
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

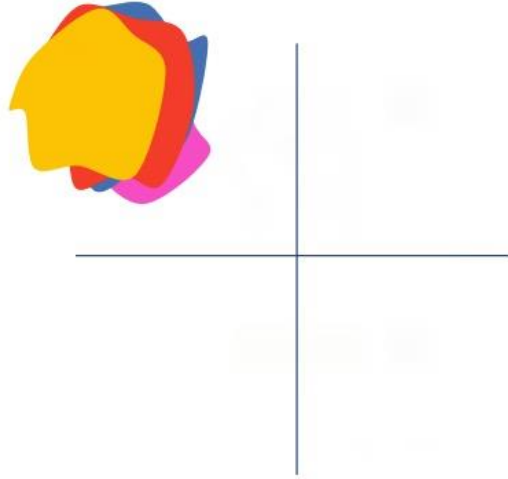
How to compute the training loss and update model weights?

On whole set once?

Batch Normalisation

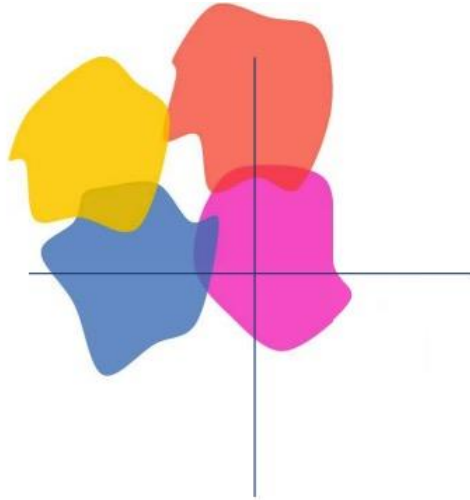


Batch Normalisation



- Training assumes the training data are all similarly distributed
 - Minibatches have similar distribution

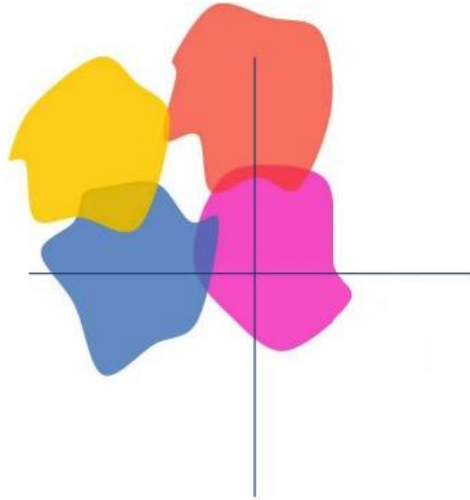
The problem of covariate shifts



What is covariate shift? Why does it make training harder?

- Training assumes the training data are all similarly distributed
 - Minibatches have similar distribution
- In practice, each minibatch may have a different distribution
 - A “covariate shift”
 - Which may occur in *each* layer of the network

The problem of covariate shifts

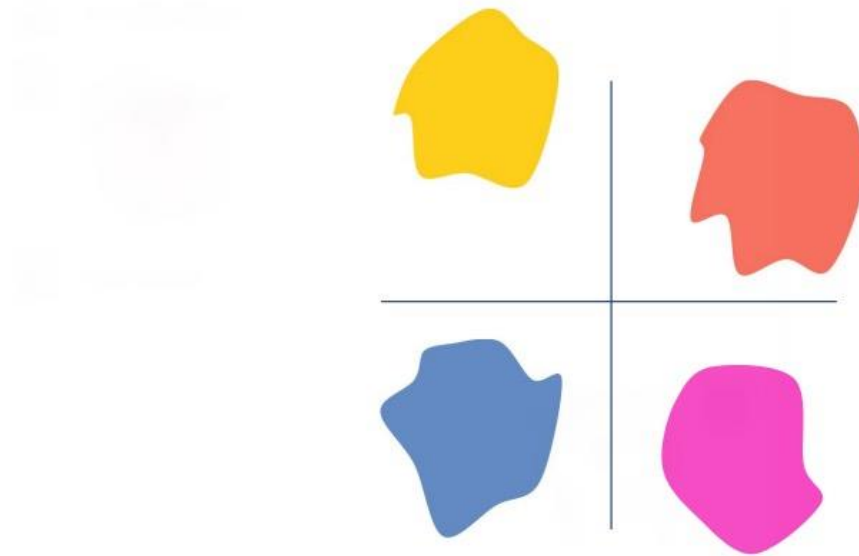


What is covariate shift? Why does it make training harder?

Change in data distribution between training and testing (or across layers). Each layer must constantly readjust to new input distributions

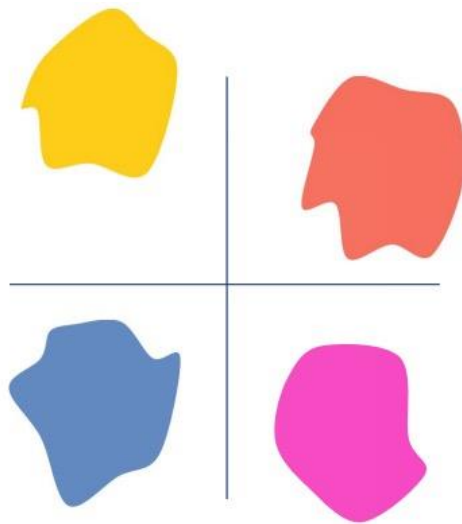
- Training assumes the training data are all similarly distributed
 - Minibatches have similar distribution
- In practice, each minibatch may have a different distribution
 - A “covariate shift”
 - Which may occur in *each* layer of the network

The problem of covariate shifts



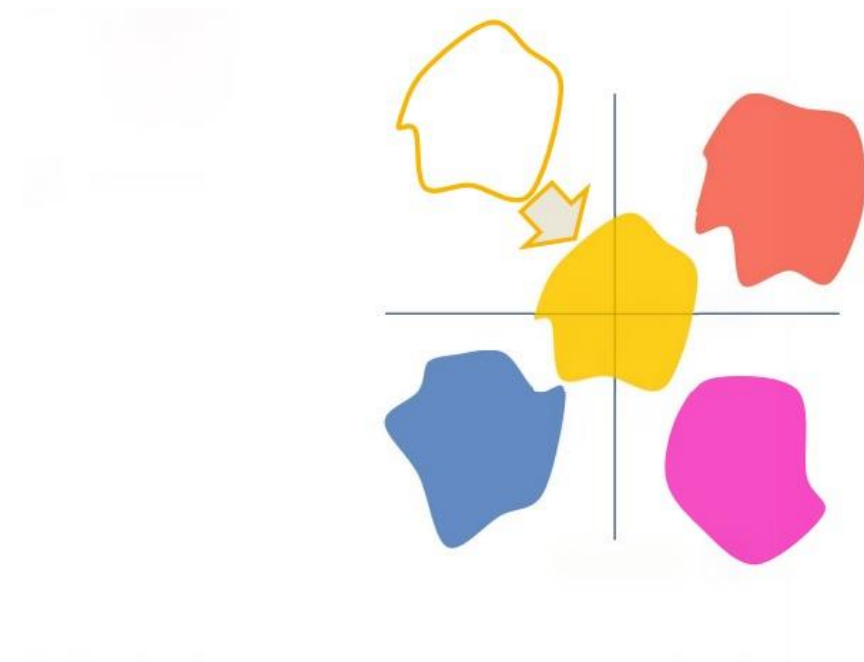
- Training assumes the training data are all similarly distributed
 - Minibatches have similar distribution
- In practice, each minibatch may have a different distribution
 - A “covariate shift”
- Covariate shifts can be large!
 - All covariate shifts can affect training badly

Batch Normalisation: Move all minibatches to a “standard” location



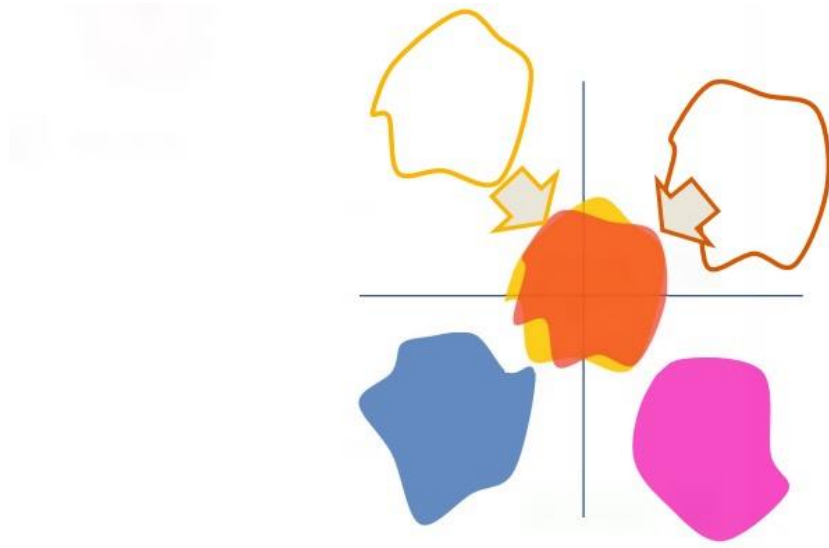
- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

Batch Normalisation: Move all minibatches to a “standard” location



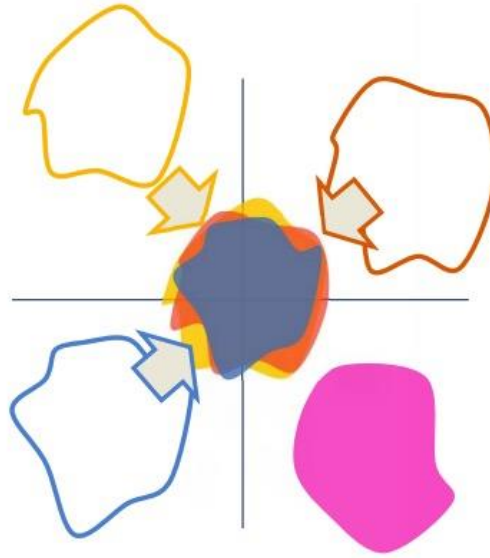
- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

Batch Normalisation: Move all minibatches to a “standard” location



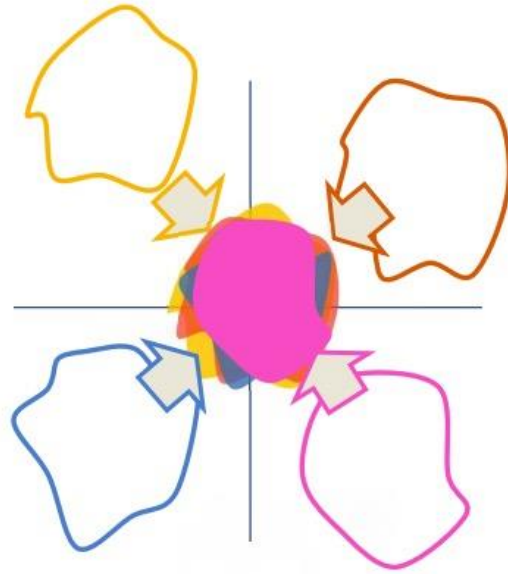
- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

Batch Normalisation: Move all minibatches to a “standard” location



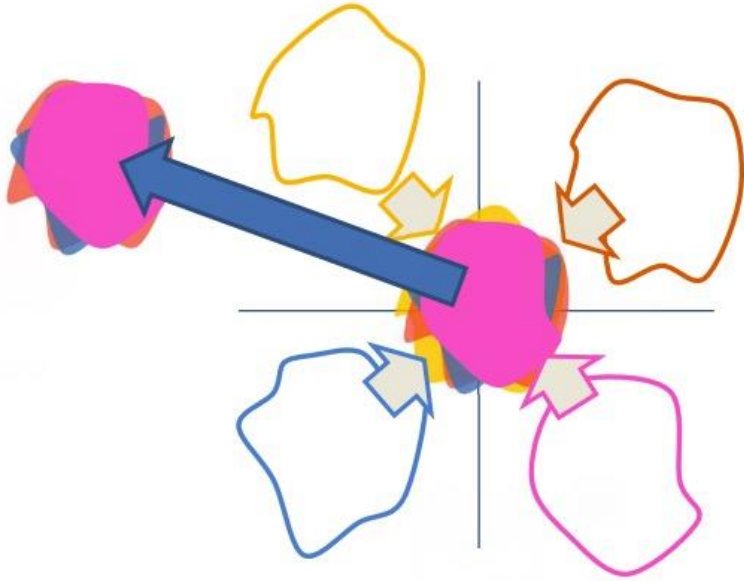
- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

Batch Normalisation: Move all minibatches to a “standard” location



- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

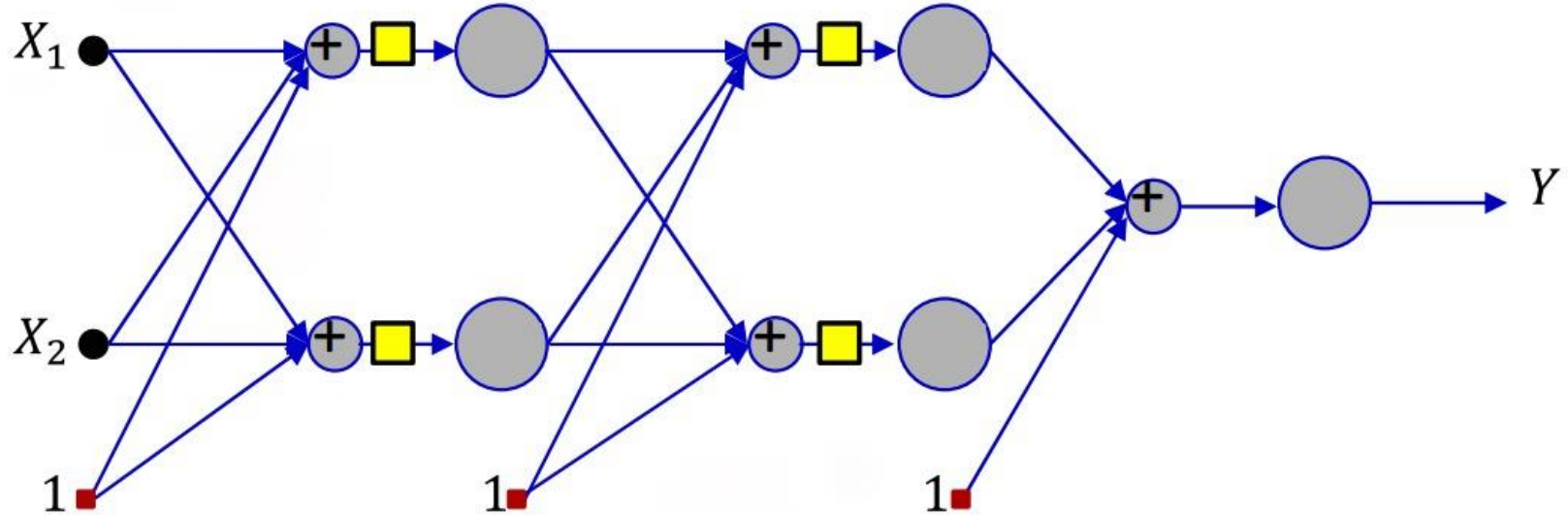
Batch Normalisation: Move all minibatches to a “standard” location



- “Move” all batches to have a mean of 0 and unit standard deviation
 - Eliminates covariate shift between batches

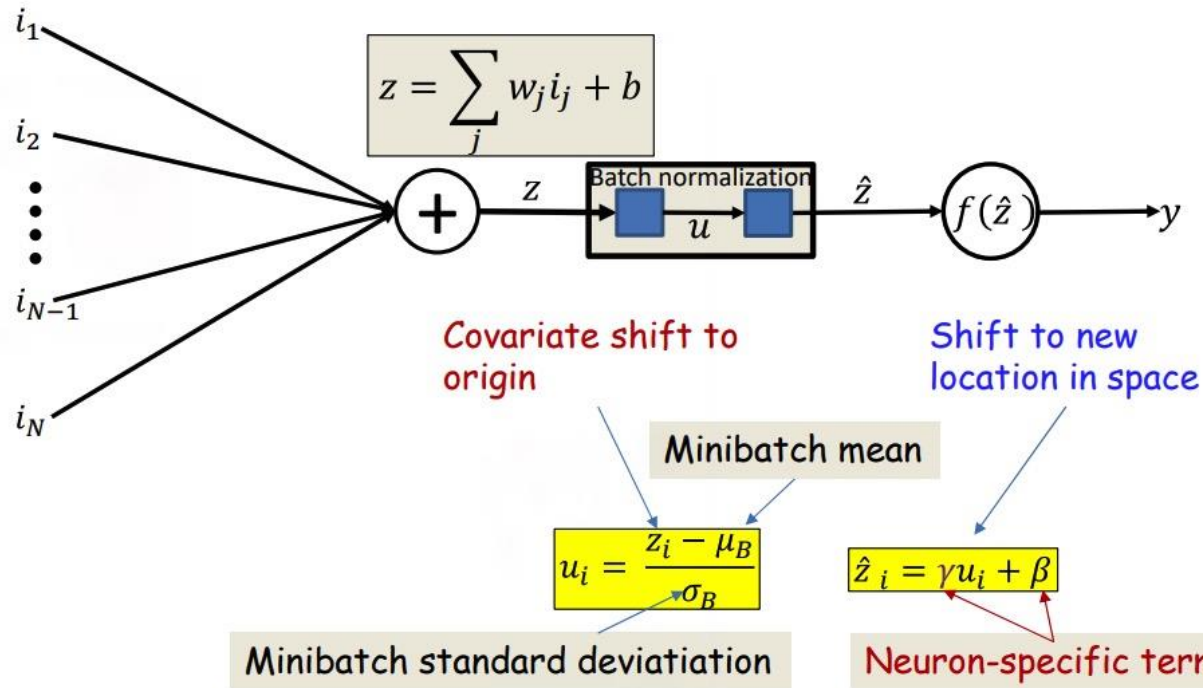
Then move the entire collection to the appropriate location

Batch Normalisation



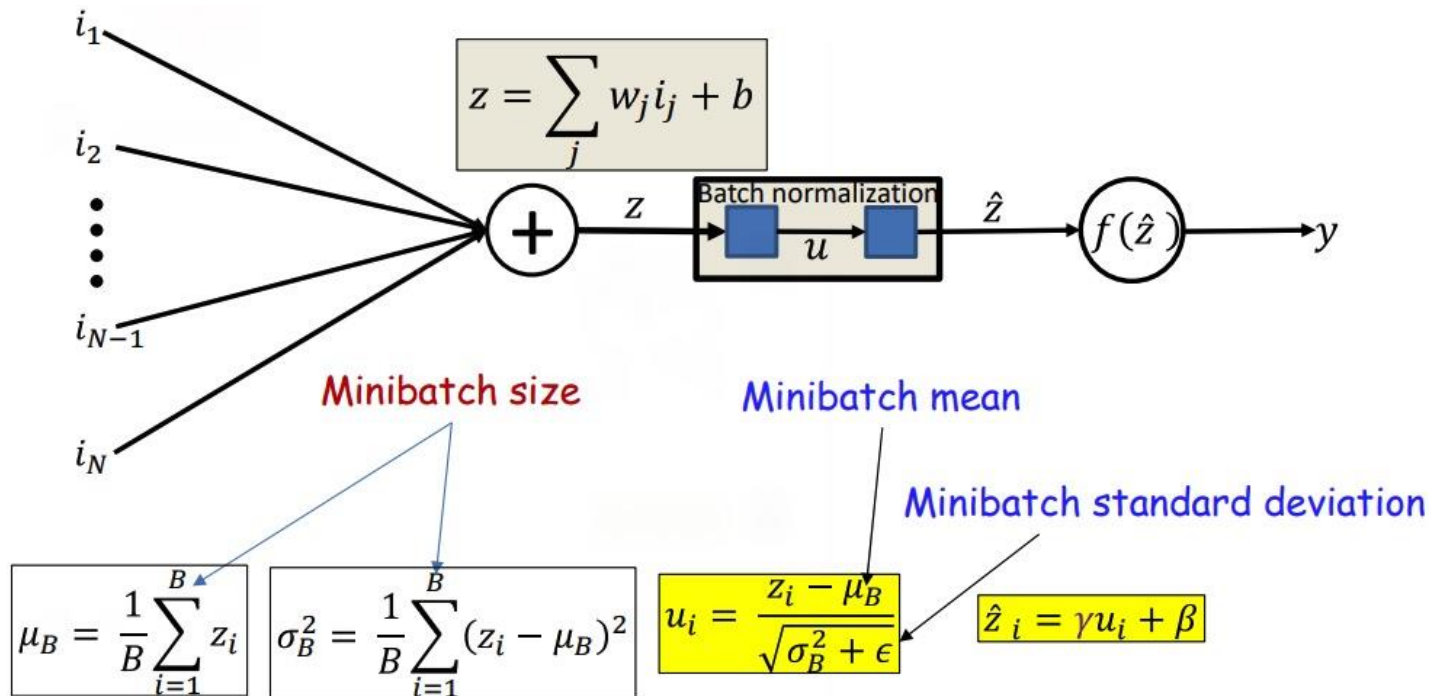
- Batch normalization is a covariate adjustment unit that happens after the weighted addition of inputs but before the application of activation
- Is done independently for each unit, to simplify computation
- **Training: The adjustment occurs over individual minibatches**

Batch Normalisation



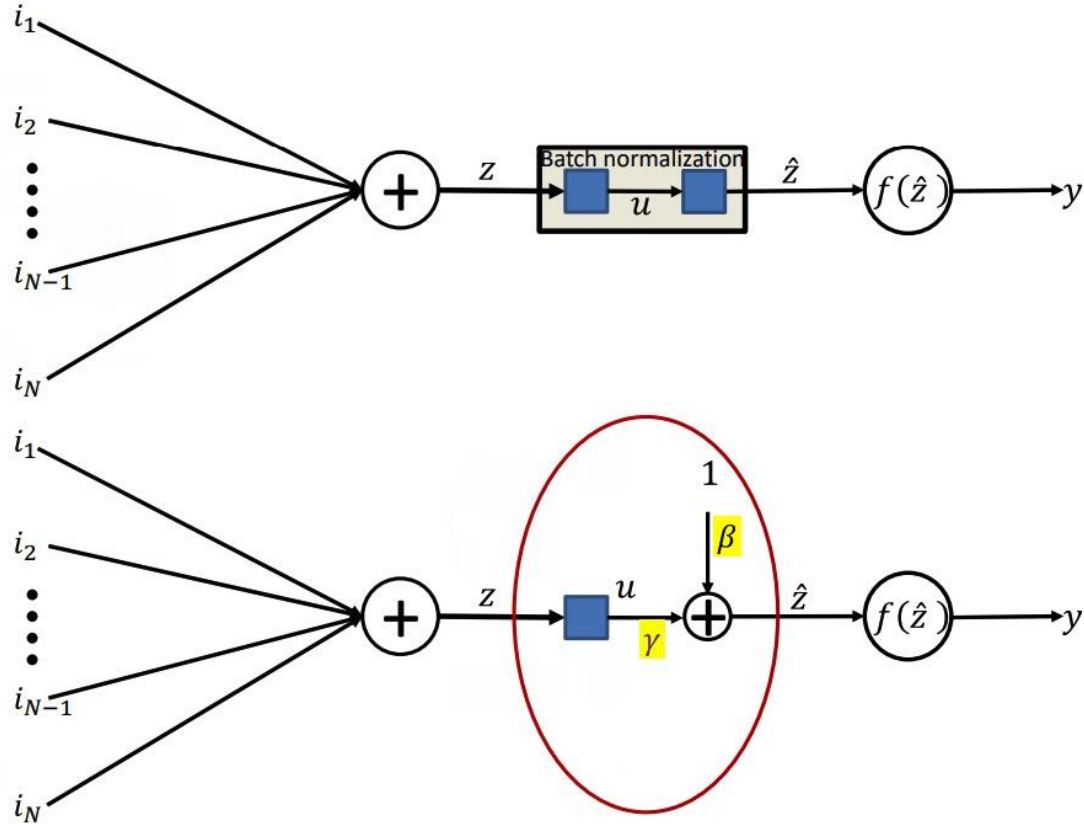
- BN aggregates the statistics over a minibatch and normalises the batch by them
- Normalised instances are “shifted” to a unit-specific location

Batch Normalisation



- BN aggregates the statistics over a minibatch and normalises the batch by them
- Normalised instances are “shifted” to a unit-specific location

Batch Normalisation



Batch Normalisation

Is BN used in test/evaluation?

Batch Normalisation

Is BN used in test/evaluation?

Yes

How?

Batch Normalisation

Is BN used in test/evaluation?

Yes

How?

It uses pre-computed, running averages of the mean and variance that were tracked during the training phase. This ensures a consistent and predictable normalisation based on the overall training data, rather than being dependent on the statistics of each individual test batch.

What would happen if we used current batch stats at test time?

Batch Normalisation

Is BN used in test/evaluation?

Yes

How?

It uses pre-computed, running averages of the mean and variance that were tracked during the training phase. This ensures a consistent and predictable normalisation based on the overall training data, rather than being dependent on the statistics of each individual test batch.

What would happen if we used current batch stats at test time?

Random output. Depends on the test batch.

Batch Normalisation

Example setup

We have a mini-batch of **4 samples** and **1 feature** (for simplicity):

$$x = [1, 2, 3, 4]$$

We'll normalise these using **BatchNorm** with learnable parameters

$$\gamma = 2, \beta = 1, \text{ and } \epsilon = 10^{-5}.$$

Batch Normalisation

◆ Step 1 — Compute batch mean

$$\mu_B = \frac{1 + 2 + 3 + 4}{4} = 2.5$$

◆ Step 2 — Compute batch variance

$$\sigma_B^2 = \frac{(1 - 2.5)^2 + (2 - 2.5)^2 + (3 - 2.5)^2 + (4 - 2.5)^2}{4} = \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = 1.25$$

Batch Normalisation

◆ Step 3 — Normalise

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\sqrt{1.25 + 10^{-5}} \approx 1.118$$

So:

x_i	Normalised $\hat{x}_i$
1	$(1 - 2.5)/1.118 = -1.342$
2	$(2 - 2.5)/1.118 = -0.447$
3	$(3 - 2.5)/1.118 = 0.447$
4	$(4 - 2.5)/1.118 = 1.342$

Batch Normalisation

◆ Step 4 — Scale and shift

$$y_i = \gamma \hat{x}_i + \beta$$

Using $\gamma = 2, \beta = 1$:

x_i	$\hat{x}_i$	$y_i = 2\hat{x}_i + 1$
1	-1.342	-1.684
2	-0.447	0.106
3	0.447	1.894
4	1.342	3.684

✓ Final output:

$$y = [-1.684, 0.106, 1.894, 3.684]$$

Batch Normalisation (PyTorch)

```
class MLP_NoBN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(1, 64),  
            nn.ReLU(),  
            nn.Linear(64, 64),  
            nn.ReLU(),  
            nn.Linear(64, 1)  
        )  
    def forward(self, x): return self.net(x)
```

No batch normalisation

```
class MLP_WithBN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(1, 64),  
            nn.BatchNorm1d(64), #  Batch Normalisation layer  
            nn.ReLU(),  
            nn.Linear(64, 64),  
            nn.BatchNorm1d(64),  
            nn.ReLU(),  
            nn.Linear(64, 1)  
        )  
    def forward(self, x): return self.net(x)
```

Apply batch normalisation layer

BN helps mainly with: (A) reducing overfitting, (B) improving optimisation, (C) both?"

Batch Normalisation (PyTorch)

```
class MLP_NoBN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(1, 64),  
            nn.ReLU(),  
            nn.Linear(64, 64),  
            nn.ReLU(),  
            nn.Linear(64, 1)  
        )  
    def forward(self, x): return self.net(x)
```

No batch normalisation

```
class MLP_WithBN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(1, 64),  
            nn.BatchNorm1d(64), #  Batch Normalisation layer  
            nn.ReLU(),  
            nn.Linear(64, 64),  
            nn.BatchNorm1d(64),  
            nn.ReLU(),  
            nn.Linear(64, 1)  
        )  
    def forward(self, x): return self.net(x)
```

Apply batch normalisation layer

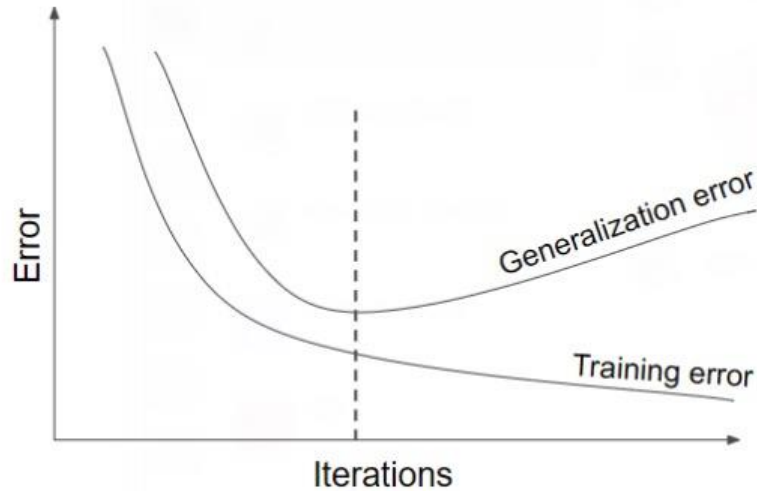
BN helps mainly with: (A) reducing overfitting, (B) improving optimisation, (C) both?"

Answer: (C) both — originally for optimisation, but it also regularises.

Batch Normalisation (Numpy)

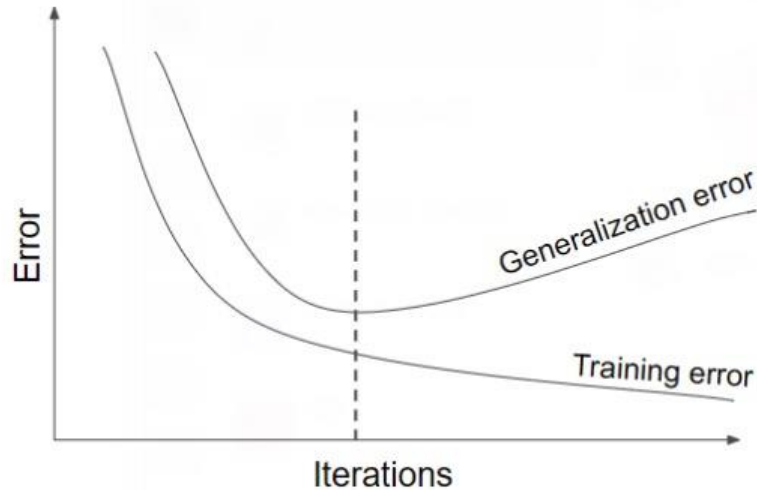
```
def batch_norm_forward(x, gamma, beta, eps=1e-5):  
    """  
    x: (N, D) batch of activations  
    gamma, beta: scale and shift parameters  
    returns: normalised output, and cache (for backprop if needed)  
    """  
    mu = x.mean(axis=0)                # batch mean  
    var = x.var(axis=0)                # batch variance  
    x_hat = (x - mu) / np.sqrt(var + eps) # normalise  
    out = gamma * x_hat + beta          # scale + shift  
    cache = (x, x_hat, mu, var, gamma, beta, eps)  
    return out, cache
```

Early Stopping



- **Know how early stopping works**
- **Understand how early stopping acts as a regulariser**

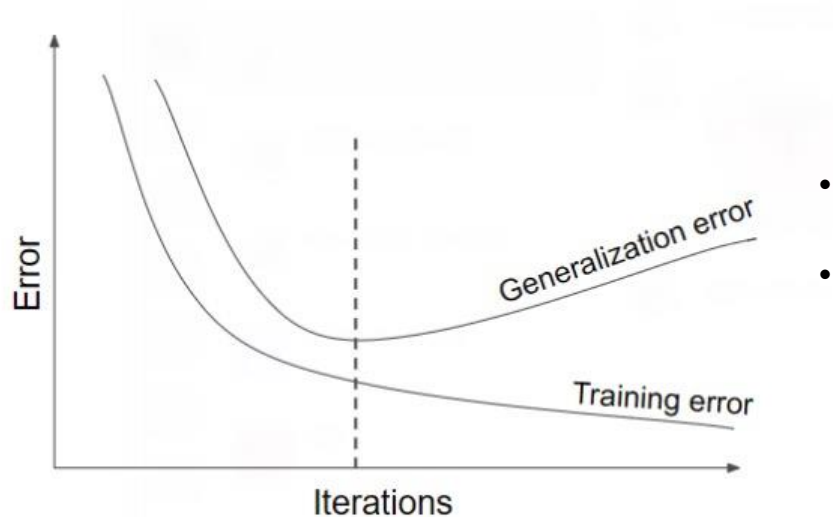
Early Stopping



- Especially for complex nonlinear models we can easily overfit
- In optimization: Often, after a certain number of iterations, generalisation error begins to increase even though training error continues to decrease

Why does validation loss increase even though training loss decreases?

Early Stopping



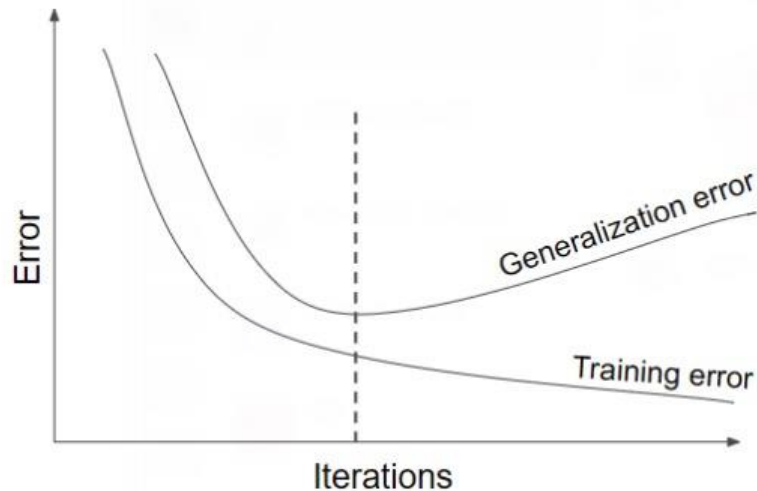
- Especially for complex nonlinear models we can easily overfit
- In optimization: Often, after a certain number of iterations, generalisation error begins to increase even though training error continues to decrease

Why does validation loss increase even though training loss decreases?

The model starts memorising training data.

Early Stopping

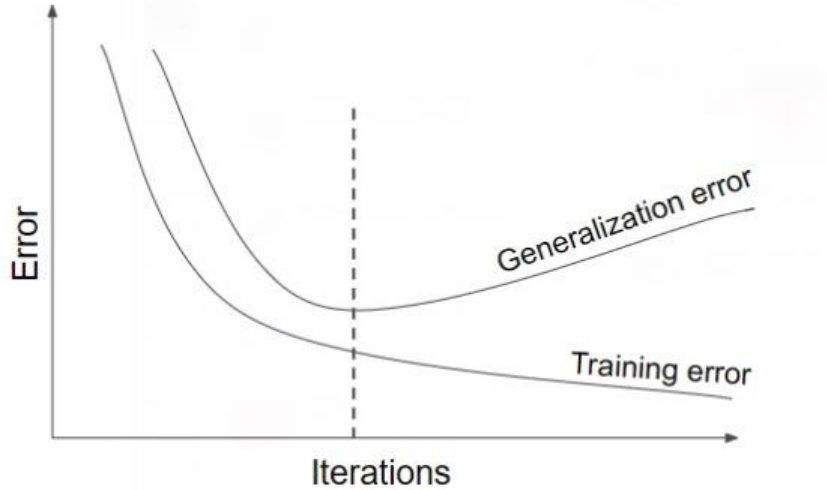
For iterative optimizers like SGD, we can monitor this step-by-step over small iterations:



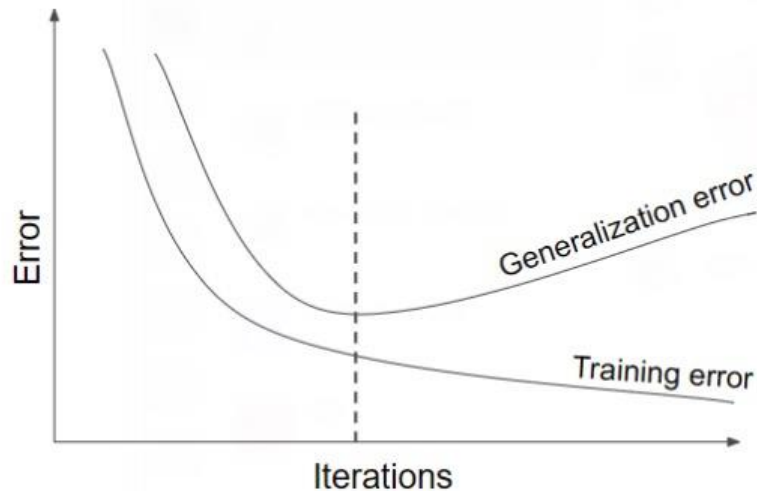
1. Split train data D_{train} into D_{subtrain} and D_{val} (e.g. with ratio of 2:1)
2. Train on D_{subtrain} and eval model on D_{val}
3. Stop when validation error stops decreasing (after a range of “patience” steps)
4. Use parameters of the previous step for the actual model

Early Stopping

Strengths



Early Stopping

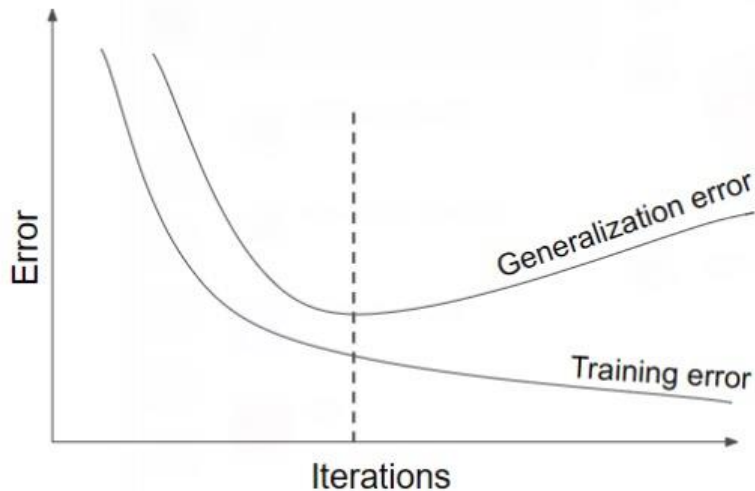


Strengths

- Effective and simple
- Applicable to almost any model without adjustment
- Combinable with other regularization methods

Weaknesses

Early Stopping



Strengths

- Effective and simple
- Applicable to almost any model without adjustment
- Combinable with other regularization methods

Weaknesses

- Periodical evaluation of validation error
- We have to save the whole model each time validation error improves
- Less data for training

Regularisers for Robustness (Concept Introduction)

Adversarial Training

Formulation:

$$\min_{\theta} \mathbb{E}_{(x,y)} \max_{\|\delta\| < \epsilon} \ell(f_{\theta}(x + \delta), y)$$

Apply adversarial perturbation on training data

Effect: Regularises model to be stable under small adversarial perturbations.

Variants: TRADES (Zhang et al., 2019), MART, AWP.

Adversarial Machine Learning



What's the difference between standard and adversarial regularisation

Regularisers for Robustness (Concept Introduction)

Adversarial Training

Formulation:

$$\min_{\theta} \mathbb{E}_{(x,y)} \max_{\|\delta\| < \epsilon} \ell(f_{\theta}(x + \delta), y)$$

Apply adversarial perturbation on training data

Effect: Regularises model to be stable under small adversarial perturbations.

Variants: TRADES (Zhang et al., 2019), MART, AWP.

Adversarial Machine Learning



What's the difference between standard and adversarial regularisation

Standard reduces loss on clean data; adversarial targets stability under attack.

Spectral and Orthogonality Regularisers (Concept Introduction)

(a) Spectral Norm Regularisation

- **Idea:** Constrain largest singular value of weights.
- **Effect:** Bounds Lipschitz constant → improves stability and robustness.
- **Use cases:** GANs (Miyato et al., 2018), certified robustness.

(b) Orthogonal Regularisation

- **Formulation:** $\|W^T W - I\|_F^2$.
- **Effect:** Encourages diversity and decorrelation among features.